

Regular Expressions and Finite Automata

1. Introduction to Regular Expressions

What are Regular Expressions?

Regular expressions (REs) are algebraic notations used to describe patterns in strings. They provide a declarative way to express what strings we want to accept in a language. Regular expressions describe exactly the regular languages, which are the same languages that can be recognized by finite automata.

Key Points:

- If E is a regular expression, then $L(E)$ represents the language it defines
- Regular expressions offer a mathematical way to describe string patterns
- They are equivalent in power to DFA, NFA, and ϵ -NFA

Example: The regular expression $01^* + 10^*$ describes strings that either start with 0 followed by zero or more 1s, OR start with 1 followed by zero or more 0s. So valid strings include: 0, 01, 011, 0111, 1, 10, 100, 1000, etc.

2. Formal Definition of Regular Expressions

Regular expressions are defined recursively with basis cases and inductive rules.

Basis Cases (Building Blocks)

Basis 1 - Single Symbol: If 'a' is any symbol from the alphabet, then 'a' is a regular expression.

- $L(a) = \{a\}$
- This means the language contains exactly one string, which is the single symbol 'a'

Example: If $a = 0$, then $L(0) = \{0\}$, which is the set containing only the string "0"

Basis 2 - Empty String: ϵ (epsilon) is a regular expression.

- $L(\epsilon) = \{\epsilon\}$
- This represents the language containing only the empty string

Basis 3 - Empty Set: $\emptyset$ (empty set) is a regular expression.

- $L(\emptyset) = \emptyset$
- This represents the empty language (no strings at all)

Inductive Rules (Combining Regular Expressions)

Induction 1 - Union (+ operator): If E_1 and E_2 are regular expressions, then $E_1 + E_2$ is a regular expression.

- $L(E_1 + E_2) = L(E_1) \cup L(E_2)$
- This represents all strings that are either in $L(E_1)$ OR in $L(E_2)$

Example:

- Let $E_1 = 0$ and $E_2 = 1$
- Then $L(0 + 1) = \{0, 1\}$
- This accepts either "0" or "1"

Induction 2 - Concatenation (• operator): If E_1 and E_2 are regular expressions, then E_1E_2 is a regular expression.

- $L(E_1E_2) = L(E_1)L(E_2)$
- This represents all strings wx where w is from $L(E_1)$ and x is from $L(E_2)$

Example:

- Let $E_1 = 01$ and $E_2 = 10$
- Then $L(0110) = \{0110\}$
- If E_1 accepts $\{0, 01\}$ and E_2 accepts $\{1, 10\}$, then E_1E_2 accepts $\{01, 010, 011, 0110\}$

Induction 3 - Kleene Closure (* operator):* If E is a regular expression, then E^* is a regular expression.

- $L(E^*) = (L(E))^*$
- This represents zero or more concatenations of strings from $L(E)$
- When $n=0$, the result is ϵ (empty string)

Example:

- $L(0^*) = \{\epsilon, 0, 00, 000, 0000, \dots\}$
- This accepts the empty string or any number of 0s

3. Operator Precedence

When writing regular expressions, operators have a specific order of evaluation:

Precedence Order (Highest to Lowest):

1. **Star (*)** - Kleene closure (highest precedence)

2. **Concatenation ($\bullet$)** - Usually written without the dot

3. **Union (+)** - Addition (lowest precedence)

Using Parentheses

Parentheses can be used to override the default precedence and group operations.

Example: $01^* + 1$

- According to precedence, this is parsed as: $(0\bullet(1^*)) + 1$
- Breakdown:
 - First, 1^* is evaluated (highest precedence)
 - Then, 0 is concatenated with 1^*
 - Finally, the union with 1 is performed
- $L(01^* + 1) = \{0, 01, 011, 0111, \dots, 1\}$

Example: $0(1+0)$

- The parentheses force union to happen first
- $L(0(1+0)) = \{01, 00\}$
- This accepts "01" or "00"
- $L(01) = \{01\}$.
- $L(01+0) = \{01, 0\}$.
- $L(0(1+0)) = \{01, 00\}$.
 - Note order of precedence of operators.
- $L(0^*) = \{\epsilon, 0, 00, 000, \dots\}$.
- $L((0+10)^*(\epsilon+1)) =$ all strings of 0's and 1's without two consecutive 1's.

4. Language Operators

Union Operation

The union of two languages L and M, written as $L \cup M$, contains all strings that appear in either L or M (or both).

Example:

- Let $L = \{0, 00\}$ and $M = \{1, 11\}$
- $L \cup M = \{0, 00, 1, 11\}$

Concatenation Operation

The concatenation of two languages L and M , written as LM or $L \bullet M$, contains all strings of the form xy where x is from L and y is from M .

Example:

- Let $L = \{0, 01\}$ and $M = \{1, 10\}$
- $LM = \{01, 010, 011, 0110\}$
- Each string in LM is formed by taking one string from L and one from M

Kleene Closure Operation (*)

The Kleene closure of a language L , written as L^* , represents zero or more concatenations of strings from L .

Formal Definition:

- $L^0 = \{\epsilon\}$ (zero concatenations gives empty string)
- $L^1 = L$ (one concatenation is just L itself)
- $L^2 = \{w_1w_2 \mid w_1 \in L, w_2 \in L\}$ (all possible pairs)
- $L^i = \{w_1w_2...w_i \mid \text{all } w_i\text{'s are from } L, \text{ duplicates allowed}\}$
- $L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$

Detailed Example: Let $L = \{1, 00\}$

- $L^0 = \{\epsilon\}$
- $L^1 = \{1, 00\}$
- $L^2 = \{11, 100, 001, 0000\}$
 - $1 \bullet 1 = 11$
 - $1 \bullet 00 = 100$
 - $00 \bullet 1 = 001$
 - $00 \bullet 00 = 0000$
- $L^3 = \{111, 1100, 1001, 10000, 0011, 00100, 00001, 000000\}$
- $L^* = \{\epsilon, 1, 00, 11, 100, 001, 0000, 111, 1100, \dots\}$

Important Note: Each w_i is chosen independently from L , and duplicates are allowed. This means we can use the same string from L multiple times.

5. Equivalence Between RE and Automata

The Big Picture

Regular expressions, finite automata (DFA, NFA, ϵ -NFA), and regular languages all describe the same class of languages. They are different representations of the same concept.

Regular Expressions $\leftrightarrow$ Finite Automata $\leftrightarrow$ Regular Languages

(Syntactic)

(Machine-based)

(Formal classes)

Two Key Theorems (Kleene's Theorem)

Theorem 1: For every DFA A , there exists a regular expression R such that $L(R) = L(A)$

- This means we can convert any DFA to an equivalent regular expression

Theorem 2: For every regular expression R , there exists an ϵ -NFA E such that $L(E) = L(R)$

- This means we can convert any regular expression to an equivalent ϵ -NFA

6. Converting Regular Expression to ϵ -NFA

General Construction Rules

When converting a regular expression to an ϵ -NFA, we follow specific patterns. The resulting ϵ -NFA has a special structure:

- **Start state:** Only state with external predecessors
- **Final state:** Only state with external successors
- No arcs come from outside or leave to outside

Basis Cases

Case 1 - Single Symbol 'a':

Start $\rightarrow [a] \rightarrow$ Final

Creates a simple two-state machine with one transition labeled 'a'.

Case 2 - Empty String ' ϵ ':

Start $\rightarrow [\epsilon] \rightarrow$ Final

Creates a two-state machine with an epsilon transition.

Case 3 - Empty Set ' $\emptyset$ ':

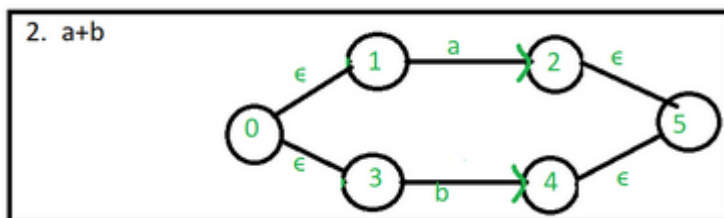
Start Final

Creates two disconnected states (no transitions, accepts nothing).

Inductive Cases

Induction 1 - Union ($E_1 + E_2$):

For the union of two regular expressions, create a new start state with epsilon transitions to the start states of both E_1 and E_2 , and epsilon transitions from both final states to a new common final state.



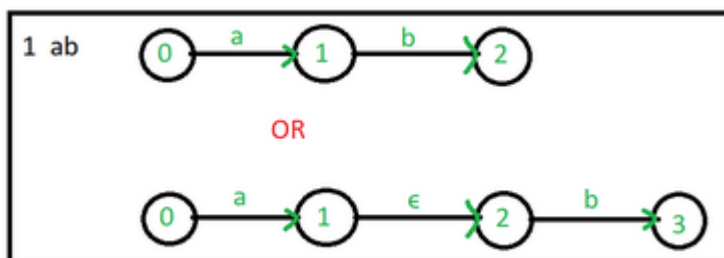
Example: For $0 + 1$

- Create paths for both 0 and 1
- Use epsilon transitions to allow choice between them

Induction 2 - Concatenation (E_1E_2):

For concatenation, connect the final state of E_1 to the start state of E_2 with an epsilon transition.

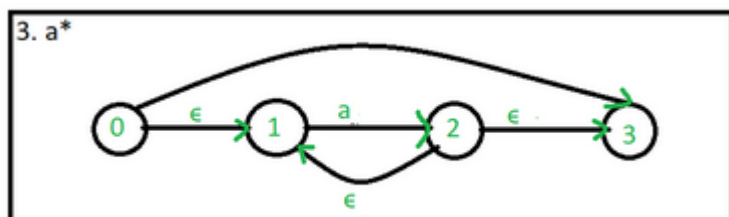
Start $\rightarrow [E_1 \text{ machine}] \rightarrow \epsilon \rightarrow [E_2 \text{ machine}] \rightarrow \text{Final}$



Induction 3 - Kleene Closure (E^*):

For closure, add epsilon transitions that allow:

- Skipping E entirely (zero repetitions)
- Repeating E multiple times



The process of converting a regular expression into an ϵ -NFA is as follows:

1. Create a single start state for the automaton, and mark it as the initial state.
2. For each character in the regular expression, create a new state and add an edge between the previous state and the new state, with the character as the label.
3. For each operator in the regular expression (such as "*" for zero or more, "+" for one or more, and "?" for zero or one), create new states and add the appropriate edges to represent the operator.
4. Mark the final state as the accepting state, which is the state that is reached when the regular expression is fully matched.

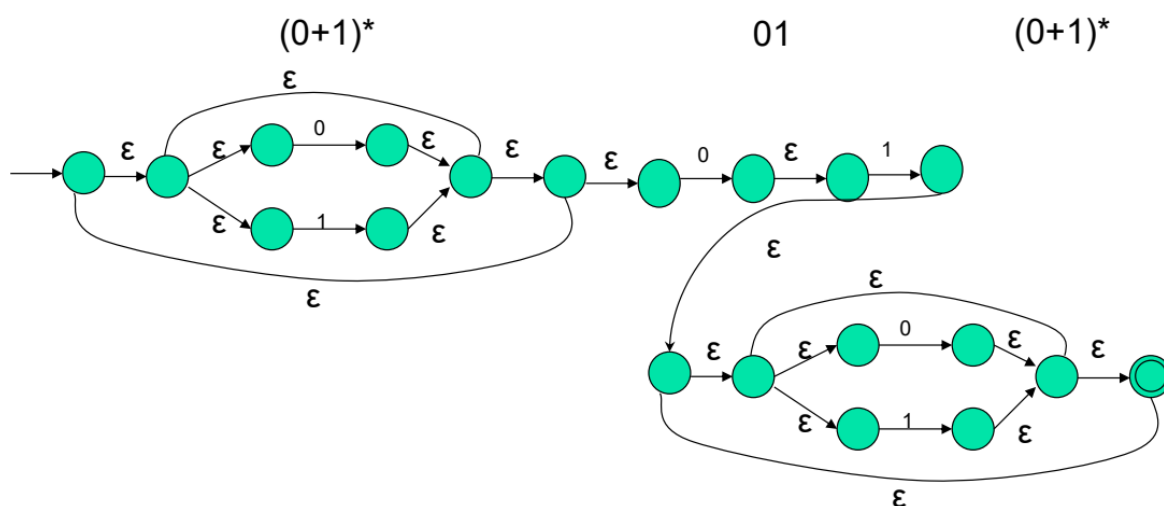
Complete Example: $(0+1)01(0+1)$

Breaking this down:

1. First part: $(0+1)^*$ - zero or more 0s or 1s
2. Middle part: 01 - literal "01"
3. Last part: $(0+1)^*$ - zero or more 0s or 1s

This regular expression accepts any binary string containing "01" as a substring.

Example: $(0+1)^*01(0+1)^*$



Meaning: The language contains all binary strings that have "01" somewhere in them.

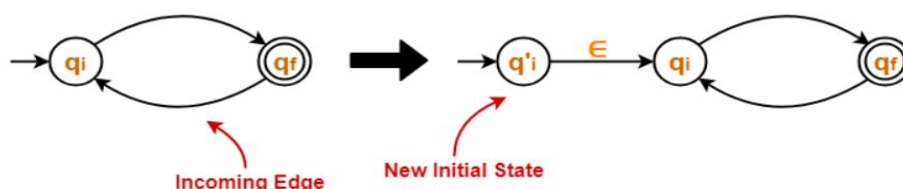
- Examples: "01", "001", "010", "101", "0011", "1101", "00100"
- Counterexample: "0", "1", "10", "110" (these don't contain "01")

7. Converting DFA to Regular Expression (State Elimination Method)

Step 1: Fix the start state

- If the start state:
 - is also a final state **OR**
 - has arrows coming **into** it
- then:
 - create a **new start state**
 - make it **non-final**
 - add an ϵ (**epsilon**) **arrow** from the new start state to the old start state

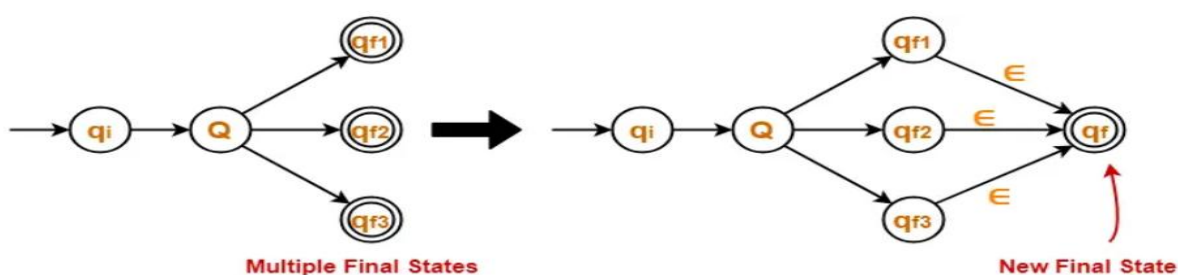
👉 This ensures the start state has **no incoming arrows**.

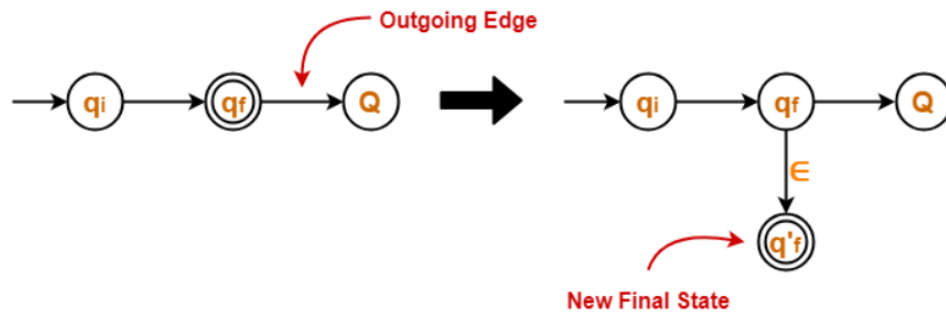


Step 2: Fix the final state

- If:
 - there are **many final states** **OR**
 - the final state has arrows going **out**
- then:
 - create **one new final state**
 - make it the **only final state**
 - change all old final states to **non-final**
 - add ϵ **arrows** from each old final state to the new final state

👉 This ensures the final state has **no outgoing arrows**.



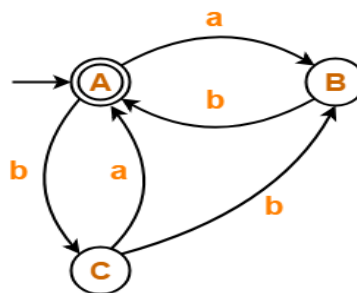


Step 3: Eliminate states one by one

- Pick a state that is:
 - **not** the start state
 - **not** the final state
- Remove that state.
- Before removing it:
 - connect its incoming and outgoing arrows properly
 - combine labels using **concatenation**, **union (+)**, and **star (*)**

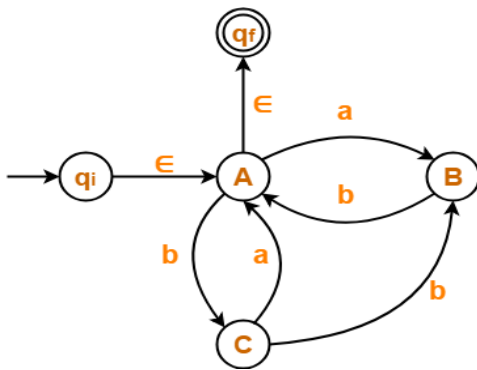
👉 This keeps the language **unchanged**.

Example-01: Find regular expression for the following DFA-

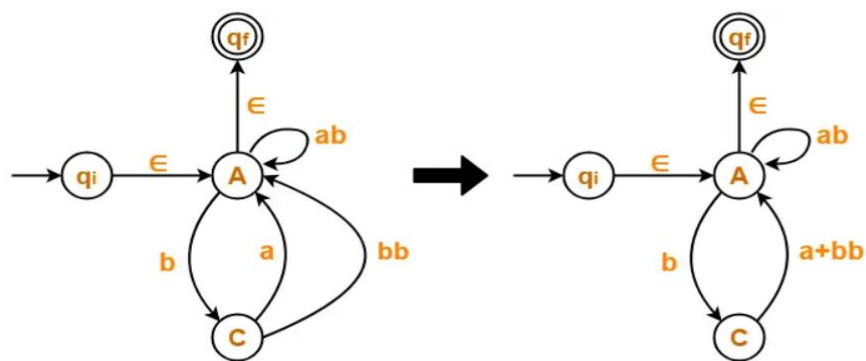


Solution-

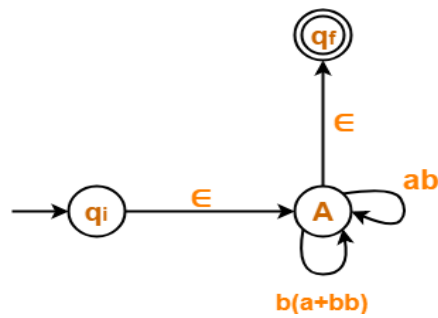
Step-01:



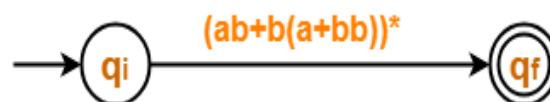
Step-02:



Step-03:

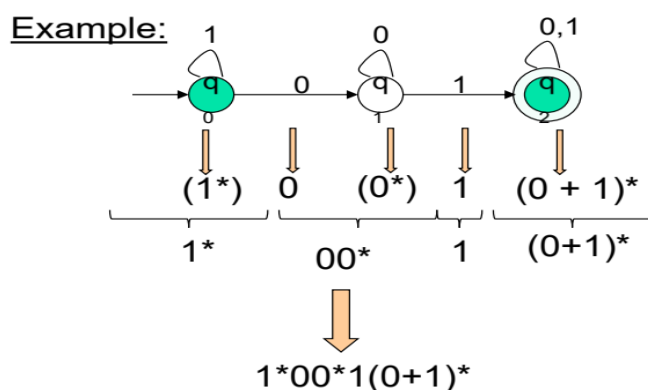


Step-04:



From here,

Regular Expression = $(ab + b(a+bb))^*$

Example-02:**8. Algebraic Laws of Regular Expressions**

These laws allow us to manipulate and simplify regular expressions.

Commutative Laws

Union is commutative: $E + F = F + E$

Example: $0 + 1 = 1 + 0$

Note: Concatenation is NOT commutative. $EF \neq FE$ in general.

- Example: $01 \neq 10$

Associative Laws

Union is associative: $(E + F) + G = E + (F + G)$

Concatenation is associative: $(EF)G = E(FG)$

Example: $((01)0)1 = (01)(01) = 0101$

Identity Elements

Identity for union is $\emptyset$: $E + \emptyset = E$ Adding the empty language doesn't change anything.

Identity for concatenation is ϵ : $\epsilon E = E\epsilon = E$ Concatenating with the empty string doesn't change the language.

Example: $\epsilon(01) = 01$

Annihilator

Annihilator for concatenation is $\emptyset$: $\emptyset E = E\emptyset = \emptyset$ Concatenating with the empty language gives the empty language.

Example: $\emptyset(01) = \emptyset$

Distributive Laws

Left distributivity: $E(F + G) = EF + EG$

Right distributivity: $(F + G)E = FE + GE$

Example: $0(1 + 0) = 01 + 00$

Idempotent Law

Union is idempotent: $E + E = E$

Example: $0 + 0 = 0$

Laws Involving Kleene Closure

- $(E^*)^* = E^*$
- $\Phi^* = \epsilon$
- $\epsilon^* = \epsilon$
- $E^+ = EE^*$
- $E? = \epsilon + E$

9. Building Regular Expressions - Complete Example

Problem Statement

Build a regular expression for the language L where:

- $L = \{w \mid w \text{ is a binary string without two consecutive 0s or two consecutive 1s}\}$

Valid strings: 0, 1, 01, 10, 010, 101, 0101, 1010, 01010

Invalid strings: 00, 11, 001, 110, 0110, 1001

Solution Approach: Case Analysis

Divide the problem based on what the string starts with and its length (even or odd).

Case A: String starts with 0 and has even length

- Pattern: 01010101...
- Regular expression: $(01)^*$
- Generates: ϵ , 01, 0101, 010101, ...

Case B: String starts with 1 and has even length

- Pattern: 10101010...
- Regular expression: $(10)^*$
- Generates: ϵ , 10, 1010, 101010, ...

Case C: String starts with 0 and has odd length

- Pattern: 010101...0
- Regular expression: $0(10)^*$
- Generates: 0, 010, 01010, 0101010, ...

Case D: String starts with 1 and has odd length

- Pattern: 101010...1
- Regular expression: $1(01)^*$
- Generates: 1, 101, 10101, 1010101, ...

Final Regular Expression

Since the language L is the union of all four cases:

$$L = (01) + (10) + 0(10)^* + 1(01)^*$$

Simplified Version

This can be simplified using the ϵ operator:

$$L = (\epsilon + 1)(01)^*(\epsilon + 0)$$

How this works:

- $(\epsilon + 1)$ at the start: optionally begin with 1
- $(01)^*$: any number of "01" pairs
- $(\epsilon + 0)$ at the end: optionally end with 0

This generates the same language more concisely.

10. Practice Problems and Verification

Problem 1: True or False

Question 1: $((R^*)) = R^*$? **Answer:** TRUE **Reason:** By the algebraic law $(E^*)^* = E^*$, applying closure multiple times doesn't change the result.

Question 2: $(R+S)^* = R^* + S^*$? **Answer:** FALSE **Reason:**

- $(R+S)^*$ includes strings that mix R and S, like RS, SRS, RRS
- $R^* + S^*$ only includes strings entirely from R^* OR entirely from S^*
- Example: Let $R = 0$, $S = 1$
 - $(0+1)^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$ (all binary strings)

- $0^* + 1^* = \{\epsilon, 0, 00, 000, \dots\} \cup \{\epsilon, 1, 11, 111, \dots\}$ (only all 0s or all 1s)

Question 3: $(RS + R)^* RS = (RRS)^*$? **Answer:** TRUE (requires proof using algebraic laws)

Problem 2: Regular Expression Examples

Example 1: $L(01) = \{01\}$

- Just the single string "01"

Example 2: $L(01 + 0) = \{01, 0\}$

- Either "01" or "0"

Example 3: $L(0(1+0)) = \{01, 00\}$

- "0" followed by either "1" or "0"

Example 4: $L(0^*) = \{\epsilon, 0, 00, 000, \dots\}$

- Zero or more 0s

Example 5: $L((0+10)^*(\epsilon+1))$ This accepts all binary strings without two consecutive 1s.

Breakdown:

- $(0+10)^*$: any sequence of "0" or "10" (ensures no "11")
- $(\epsilon+1)$: optionally end with "1"
- Valid: 0, 1, 00, 01, 10, 010, 100, 101, 1010
- Invalid: 11, 011, 110, 111